A Small Clinic Business

Adriel Nguyen

Table of contents:

- System Architecture and System Design
- System Implementation Planning
- Algorithms and Data Model Implementation
- User Interface Design and Implementation
- Design of tests & Software testing
- Error Handling
- Collaboration and Code Integration
- Debugging & Logging
- Performance Optimization Plan
- Maintenance and Support Strategy

User problem 1: There is no way to control how user allow / not allow to do with important decision making

User problem 2: There is no way to interact between user with the nurse or nurse with the doctor

User problem 3: There is no way to keep track of what each user do or log it and saves in real time using database system

Functional Features:

- RBAC (clear role-based access) with clear what each user must do and what can they allow / not allowed to do
- Real time chat between users
- Audit logs save what each user do and database to record the history

System Architecture and System Design

System Architecture and User Flow Diagram

Patient

- Create an account
- Login / logout to the system
- Chat with Nurse / Doctor
- Schedule appointment with doctor
- Pay billing

Nurse (prescription)

- Create an account
- Login / logout to the system
- Gives medicine prescription

Receptionist (billing)

- Create an account
- Login / logout to the system
- View Patient Billing
- Confirm patient billing match with the diagnosis service and medicine bills

Doctor (health diagnosis)

- Create an account
- Login / logout to the system
- Schedule with patient,
- Add doctor note
- Can export documents (with permission)

System Implementation Planning

- We are building a web application that connects both back/front end together
- It has login, UI components, database, and scheduling
- Our architecture begins with foundation with system data model and logic validation
- Each part of the system communicates together through API

What could go wrong with the system?

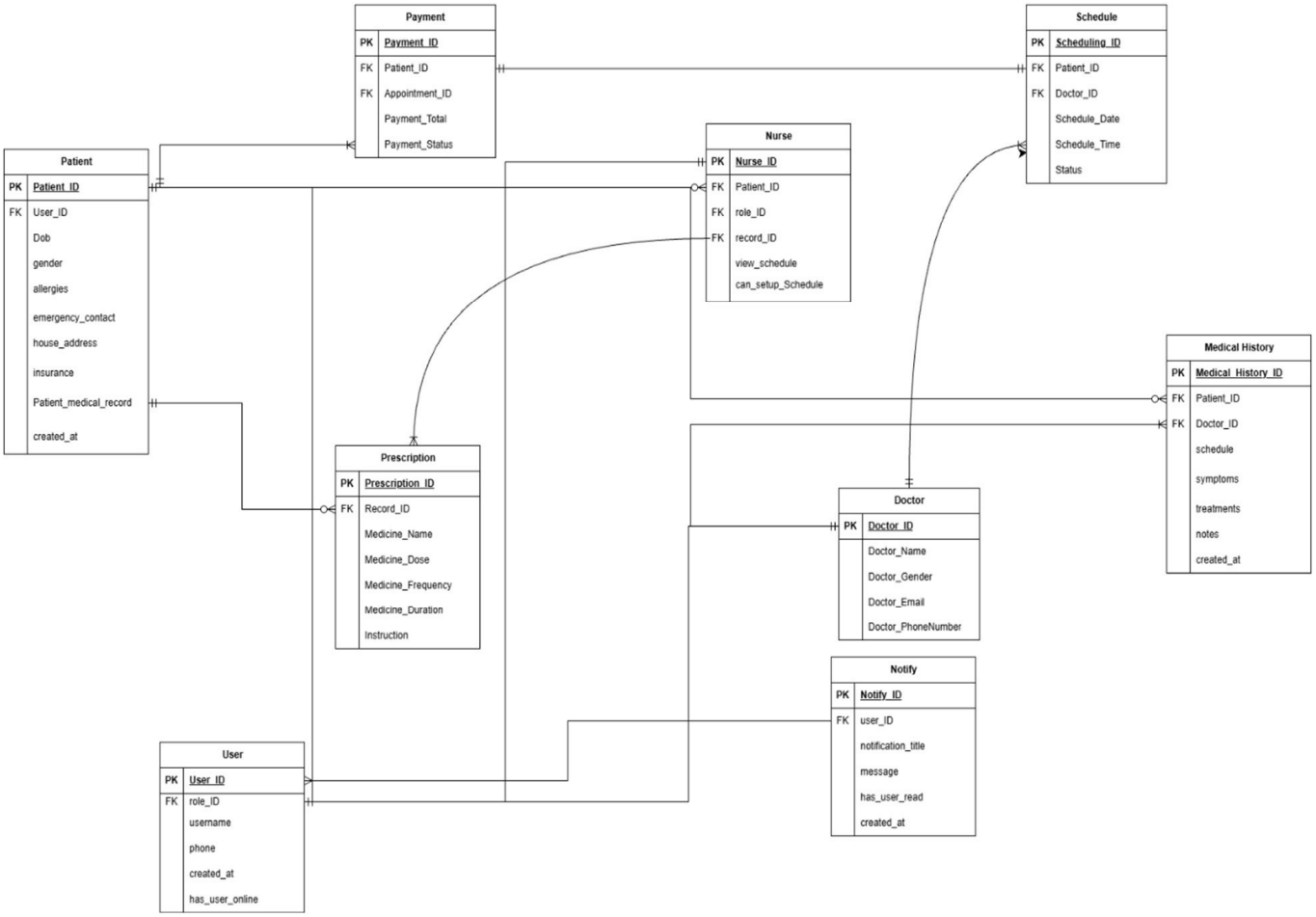
Systems don't store patient information

System doesn't track the medicine prescription that the patients are use

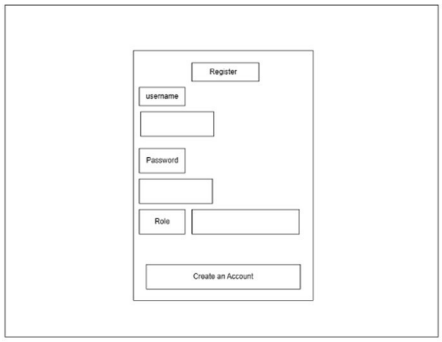
How do we fix the things that might go wrong with the system?

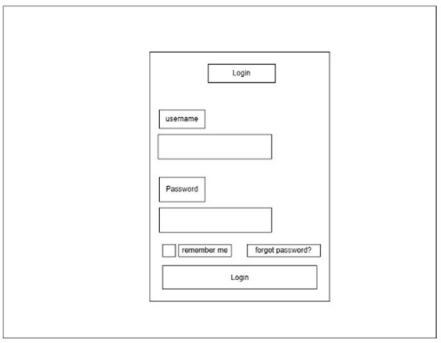
- Audit log
- Role base access
- Check if test cases / edge case pass with unit / system test/ integrated test

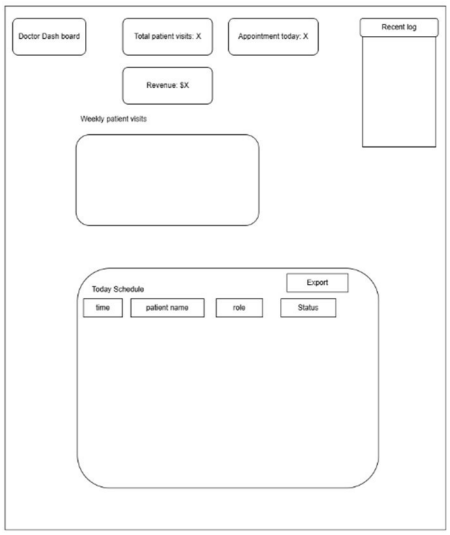
Algorithms and Data Model Implementation

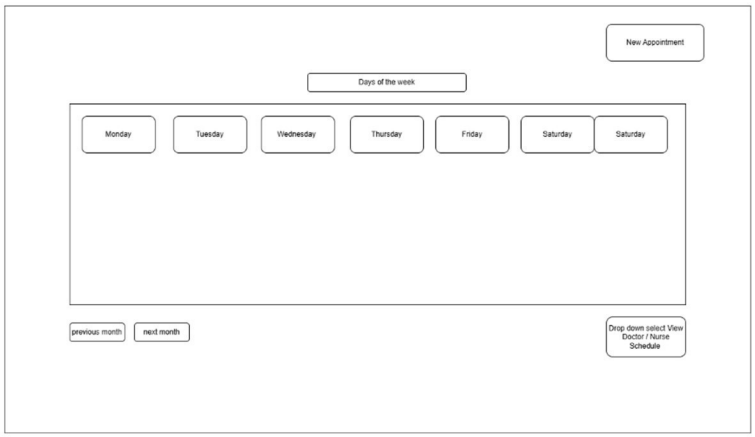

Payment
PK Payment_ID
FK Patient_ID
FK Appointment_ID
Payment_Total
Payment_Status
Schedule
PK Scheduling_ID
FK Patient_ID
FK Doctor_ID
Schedule_Date
Schedule_Time
Status
Nurse
PK Nurse_ID
FK Patient_ID
FK role_ID
FK record_ID
view_schedule
can_setup_Schedule
Patient
PK Patient_ID
FK User_ID
Dob
gender
allergies
emergency_contact
house_address
insurance
Patient_medical_record
created_at
Medical History
PK Medical_History_ID
FK Patient_ID
FK Doctor_ID
schedule
symptoms
treatments
notes
created_at
Prescription
PK Prescription_ID
FK Record_ID
Medicine_Name
Medicine_Dose
Medicine_Frequency
Medicine_Duration
Instruction
Doctor
PK Doctor_ID
Doctor_Name
Doctor_Gender
Doctor_Email
Doctor_PhoneNumber
Notify
PK Notify_ID
FK user_ID
notification_title
message
has_user_read
created_at
User
PK User_ID
FK role_ID
username
phone
created_at
has_user_online

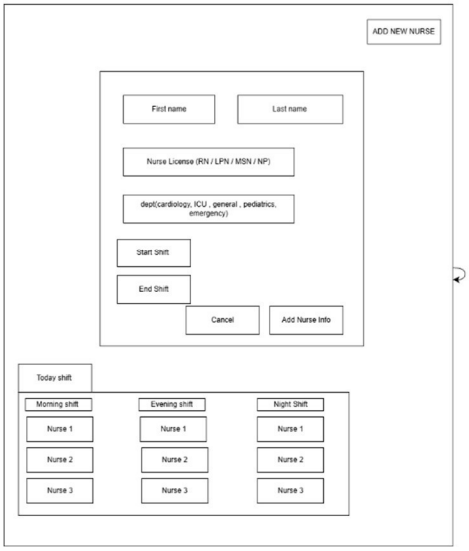
User Interface Design and Implementation











Design of tests & Software Testing

Unit test design

Visit unit test 1: user privilege cannot change (visit) state, expect system returns errors

Visit unit test 2: visit must exist to continue, expect system flag cannotProgress(null) = false

Visit unit test 3: Assign role for patient must be at least one doctor or one nurse and one patient, otherwise return need to recheck (minimum expert need for operation)

Visit unit test 5: Patient visit cannot skip states, expect system flag error when create appointment to completed

Billing unit test 1: No visit, no billing sends to the patient (no charge patient randomly)

Expect system: billing(null) = false

Billing unit test 2: No completion (on doctor visit), no billing sent back to user

Billing unit test 3: Clinic Fee and Service must be > 0

Expect system and logic use to check fee ≤ 0, throw exceptions / errors when detect

Billing unit test 4: No alteration of billed visit (for example: patient already paid for visit; the member can't change the number anymore).

Expect system: not allow to alter. Reasons behind it, if someone tried to change or resend the bill, the system not allowed to enter because the process is already completed.

Logic: rebilling, expect system to return error

Integration test

System test (communication)

Message belongs to visit only

Expect: message (no visit)
Goal and logic behind it: separate each patient in many different visits

Visit must be active

Goal: message (completed)

Sender must be a real user

System throws errors when an outside user (not logged in) try to access the app

Document control & System handling

Doctor is the only person who may export

Logic use: if export (user_role = nurse) then error

Export must require completed visit

Logic use: if export (state = in-progress) not allow

Edge case: user try to change the report.

Logic use: set state where export is read only by edit(export) sends error message

Role and action the users are allow to do

Patient:

Can chat

Cannot Export

Cannot change Billing number

Nurse :

Can chat

Cannot Edit (change) Doctor diagnosis

Cannot export Doctor diagnosis patient medical history

Doctor:

Can complete visit

Doctor:

Can export

Create

Check-in: allow (chat)

In-progress: allow (chat)

Complete: allow view(Billing) and export(complete) – for doctor

Bill: allow view and export (for reception)

Roles {patient, nurse, doctor}

Error Handling

Edge case 1: Ignore message before joinVisit , ignore everything before it (apply for web socket)

Explain logic: before joining a room must need room_id, doctor / nurse permission and must be a real user (sends error)

Edge case:

Nurse pretends to be the doctor to give diagnosis

Explain logic: Block from the start, and sends error in audit log

Edge case:

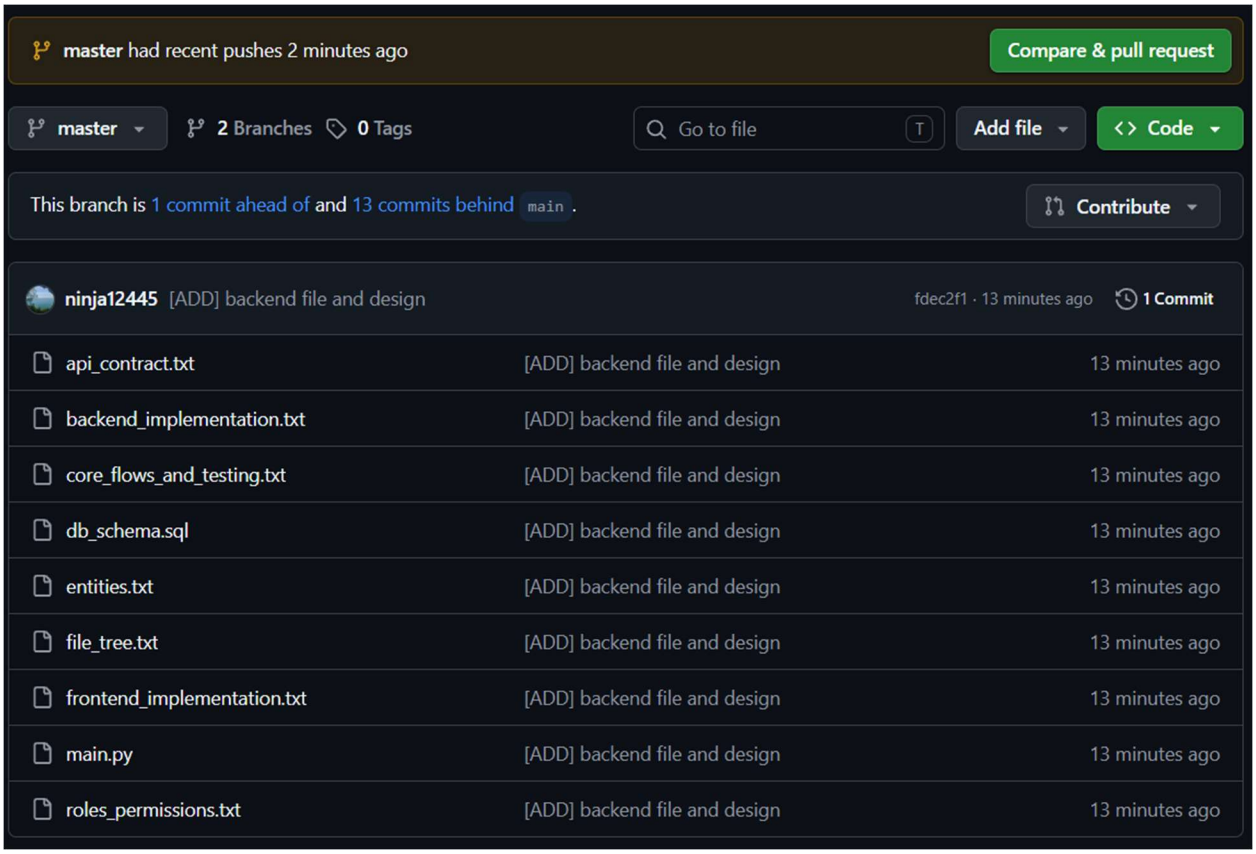
Billing twice when user click

Each time the system sends billing, only allow one click prevent over charge patient.

Edge case handle UI bugs, API misuse and Privacy

Collaboration and Code Integration

Git version control using git merge, branch and push (from local to GitHub web)



Git

```
$ git clone https://github.com/ninja12445/small-clinic-business.git
Cloning into 'small-clinic-business'...
remote: Enumerating objects: 42, done.
remote: Counting objects: 100% (42/42), done.
remote: Compressing objects: 100% (34/34), done.
remote: Total 42 (delta 10), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (42/42), 13.20 KiB | 270.00 KiB/s, done.
Resolving deltas: 100% (10/10), done.
```

```
git init
```

```
Initialized empty Git repository
```

```
git commit -m "[ADD] backend file and design"
```

```
[master (root-commit) fdec2f1] [ADD] backend file and design
 9 files changed, 566 insertions(+)
 create mode 100644 api_contract.txt
 create mode 100644 backend_implementation.txt
 create mode 100644 core_flows_and_testing.txt
 create mode 100644 db_schema.sql
 create mode 100644 entities.txt
 create mode 100644 file_tree.txt
 create mode 100644 frontend_implementation.txt
 create mode 100644 main.py
 create mode 100644 roles_permissions.txt
```

```
git remote add origin https://github.com/ninja12445/small-clinic-business.git
```

```
git remote -v
```

```
origin  https://github.com/ninja12445/small-clinic-business.git (fetch)
origin  https://github.com/ninja12445/small-clinic-business.git (push)
```

```
git branch
```

```
* master
```

```
On branch master
nothing to commit, working tree clean
```

```
git push -u origin master
```

```
info: please complete authentication in your browser...
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 12 threads
Compressing objects: 100% (11/11), done.
Writing objects: 100% (11/11), 6.25 KiB | 639.00 KiB/s, done.
Total 11 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'master' on GitHub by visiting:
remote:      https://github.com/ninja12445/small-clinic-business/pull/new/master
remote:
To https://github.com/ninja12445/small-clinic-business.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
```

gitlog

```
fdec2f1 (HEAD -> master, origin/master) [ADD] backend file and design
```

Debugging & Logging

Performance Optimization:

- Efficient algorithm
- Database indexing
- Caching
- Lazy loading

Performance Optimization Plan

- It takes longer to process when taking 50+ users sending requests (at the same time) in real time
- Authentication and user role validation, the issue found is when login it might be getting slower
- UI & UX design so inefficient, it takes more clicking time for user to figure out, maybe it harder to find certain features and there is no notification or anything to show that the system is working while let user waits too long
- Design bad architecture upfront, why? Because when try to fix one single feature, it keeps sending mistakes and problems in 2 or 3 other places.

Maintenance & Support Strategy

Post-deployment support

1. Pre-roll out demo

2. Patch with user request with report issue

3. Enables hardware setup and use network layer to ensure the system is secure

Adaptive and Perfective

1. Ensure the system can help the doctor with the scheduling problem, with nurse export issue and resending password and access for login
2. Minimal bugs and errors
3. Check system safety to the user
4. Maintain reliability in terms of the system provide a useable and reliable features

Requirement: when deliver demo must explain what these features do and how does each components / feature are used and the explanation must be easy to understand.

When deploy on Backend

```
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+
C to quit)
INFO:     Started reloader process [1964] using WatchFiles
INFO:     Started server process [6008]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     127.0.0.1:62533 - "GET /user HTTP/1.1" 200 OK
INFO:     127.0.0.1:56994 - "GET /user HTTP/1.1" 200 OK
WARNING:  WatchFiles detected changes in 'main.py'. Reloading..
```

When deploy on the Frontend

```
PS                                                    npm run dev

> small-clinic-project@0.1.0 dev
> next dev

▲ Next.js 16.2.4 (Turbopack)
- Local:        http://localhost:3000
```